

Overlapping MoE Synchronization and Data Transfer with In-Network Computing

Abstract

Some expert-parallel communication paths use a pre-barrier before payload injection and a post-barrier after sender-side transfer work. At small batches, these stages can occupy a substantial fraction of the operator. We study cross-node DeepEP V2 Direct Dispatch and Combine on two H200 nodes. At eight tokens per rank and EP8, the pre-barrier occupies about 20% of the traced operator. In Qwen3-30B-A3B decode, the pre-barrier accounts for 15.2–16.7% of the sampled cross-node GPU step, compared with 7.9–9.3% within one node.

We propose RendezEP, a design that coordinates these transfer boundaries in the network. A rank announces readiness and uploads directly into network staging; the network forwards data once the required ranks and destination buffers are ready. At completion, the expected receive count lets the network send DONE after the last data for a destination. We specify the visibility, buffer-lifetime, and progress requirements for these operations and analyze their effect on the communication critical path. Under a symmetric reference schedule with an exposed control traversal at each boundary, the opportunity approaches one fabric RTT before added overhead. The measurements characterize the baseline; the design’s net benefit remains to be evaluated on an implementation.

CCS Concepts: • Networks → Network protocol design; In-network processing; • Computer systems organization → Distributed architectures.

Keywords: expert parallelism, mixture-of-experts, in-network computing, synchronization, collective communication, datacenter networks

1 Introduction

Mixture-of-experts (MoE) inference uses Dispatch to send token activations to selected experts and Combine to return their outputs. Expert-parallel (EP) communication libraries optimize this movement through GPU-initiated transport, staging buffers, and topology-aware transfers [3, 5]. Small batches also make control latency significant: a sender may wait for peer readiness before sending an available token, and a receiver may wait for a completion notification before using delivered data.

We focus on cross-node DeepEP V2 Direct Dispatch and Combine. In this implementation, a pre-barrier precedes payload injection. At the other boundary, sender completion and local coordination precede the post-barrier signal exchange across the EP group [4]. This placement serializes parts of

coordination with data movement. It is a property of the chosen path; other EP implementations have different readiness and completion protocols.

Measurements on two H200 nodes show why these boundaries matter. At EP8 and eight tokens per rank, the pre-barrier occupies about 20% of the traced operator. Increasing the token count to 128 reduces its share to about 5%, while the absolute pre-barrier duration remains close to 15 μ s. In Qwen3-30B-A3B decode at fixed EP8, the pre-barrier accounts for 15.2–16.7% of the sampled cross-node GPU step, compared with 7.9–9.3% on one node. These measurements motivate the question: can pre-barrier and post-barrier communication overlap data transfer while preserving the conditions required by the consumer?

We propose RendezEP, which places coordination at a network node on the data path. Each rank announces readiness and uploads directly into network staging. The node aggregates readiness while data is in flight, buffers early data, and forwards it when the destination buffers are writable. It also tracks the expected receive count for each destination and sends ordered DONE after its last data.

This organization preserves two separate obligations. The receiver must obtain complete, visible input before computation begins, and buffers cannot be reused until the last computation using them finishes. Moving target notification into the network does not remove the need to retain source buffers during transmission or genuine global dependencies. Existing multicast and reduction can be used in the data plane; their bandwidth benefits are separate from the timing changes studied here.

The analysis follows readiness, data visibility, and the start of dependent computation as distinct events. We borrow Swift’s separation of endpoint and fabric delay [9] to avoid treating a measured GPU barrier as a network RTT. Unequal readiness times, finite staging, and output service determine whether early upload advances completion. At the tail, the benefit depends on the exposed interval between destination data visibility and baseline notification. In a symmetric reference where each boundary adds one exposed one-way traversal, the two opportunities sum to one fabric RTT before the design’s added cost.

This paper makes three contributions. First, it measures the pre-barrier and the post-barrier signal window of cross-node Direct, including pre-barrier costs in a real MoE serving stack. Second, it specifies network readiness aggregation and ordered completion notification with explicit resource and visibility requirements. Third, it provides a latency model for identifying when these mechanisms shorten the critical

path. The evaluation characterizes existing software baselines; it does not include an INC prototype or measured INC speedup.

2 Background and Motivation

2.1 Expert-Parallel Communication

An MoE layer selects up to $topk$ experts for each input token. Under expert parallelism, experts are distributed across GPU ranks. Dispatch moves activations to the selected expert ranks; after expert computation, Combine returns and merges their contributions at the token owner [3, 5].

Routing determines the amount of traffic sent to each destination. Communication therefore involves both activations or partial outputs and metadata describing destinations, valid counts, and output placement. Within a node, ranks may use NVLink; across nodes, they use a network transport such as RDMA. Communication staging provides known landing addresses, while the final expert layout may depend on exchanged counts.

Ranks exchange routing metadata, valid counts, and readiness or completion signals through collective operations or peer-to-peer writes and polling. Readiness signals indicate that resources are ready; completion signals indicate that the associated transfers have finished.

EP implementations organize these exchanges differently. NCCL EP HT gathers routing metadata before Dispatch; DeepEP V2 Direct performs a pre-barrier, then processes counts alongside data; NCCL EP LL writes into reserved slots and uses fine-grained signals without an independent pre-barrier [4, 12, 13]. These schedules must all ensure writable receive resources, visible input before consumption, and safe buffer reuse after consumption.

2.2 Synchronization in DeepEP V2 Direct

Cross-node DeepEP V2 Direct places coordination at both ends of Dispatch and Combine [4]. A pre-barrier precedes payload injection. Dispatch exchanges counts and computes layout alongside data movement; Combine reuses the established routing. After completing the required sender-side work, ranks exchange completion signals across the EP group in a post-barrier.

The Direct path preallocates communication buffers, and changes to its synchronization must preserve safe buffer reuse and data visibility. Opportunities to overlap pre-barrier and post-barrier communication with data transfer exist within or across nodes, provided the relevant traffic passes through INC and data-before-DONE ordering holds.

At fixed EP size, fewer tokens reduce payload work while coordination still involves the same ranks. Our EP8 measurements show a pre-barrier duration of approximately 15 μ s, occupying about 20% of the operator at eight tokens per rank and 5% at 128. This motivates overlap, although

the measured interval is not entirely removable. Section 5 presents the methodology and results.

2.3 Opportunities for In-Network Computing

In-network computing (INC) allows switches and other network devices to compute while forwarding data, for example by summing values from multiple GPUs. These devices can also retain temporary state across packets, such as which ranks are ready and how much data has been forwarded to each destination. Prior work uses in-network computation for collective reduction and data transfer in MoE communication [16–18]. We use this state to aggregate readiness signals and track completion, together with network buffering to overlap synchronization with data transfer.

A locally ready rank can begin uploading to network staging while INC collects readiness signals from other ranks. Destination delivery still waits for the required readiness and writable resources. At completion, INC can track expected output and append an ordered notification to the destination’s final data. The notification must imply data visibility, and genuine global dependencies remain.

Key insight. Synchronization conditions must hold, but their communication need not be serialized with data transfer. INC can overlap readiness aggregation with upload and append ordered completion notifications to trailing data, potentially shortening the critical path.

3 Design

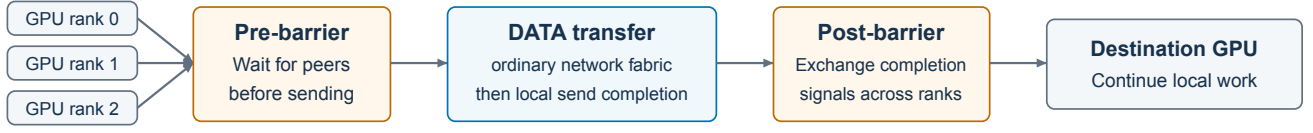
RendezEP collects readiness signals while ranks upload data and notifies each destination when its input is complete.

3.1 Overview and Assumptions

Figure 1 shows the two changes: uploading while collecting READY and sending DONE after the last data for each destination. INC tracks (i) which ranks are ready, (ii) how many tokens or reduced results each destination should receive, (iii) how many have been forwarded, and (iv) which communication round uses each receive buffer. It also buffers early-arriving data. A generation identifies a Dispatch or Combine round; a buffer epoch distinguishes successive uses of the same buffer. We assume reliable, ordered delivery: a destination observes DONE only after the associated data is visible in its GPU memory.

Receive buffers are allocated and registered symmetrically during initialization. INC obtains their layout, address mappings, sizes, and access permissions once. Each token carries routing information and an identifier used to compute its destination buffer offset without global receive counts. READY confirms that the previous round has finished using the receive buffer and that it is writable for the current round. Sources upload directly; PFC pauses DATA transferring when network buffers approach capacity.

A Direct baseline



B Our design: in-network computing

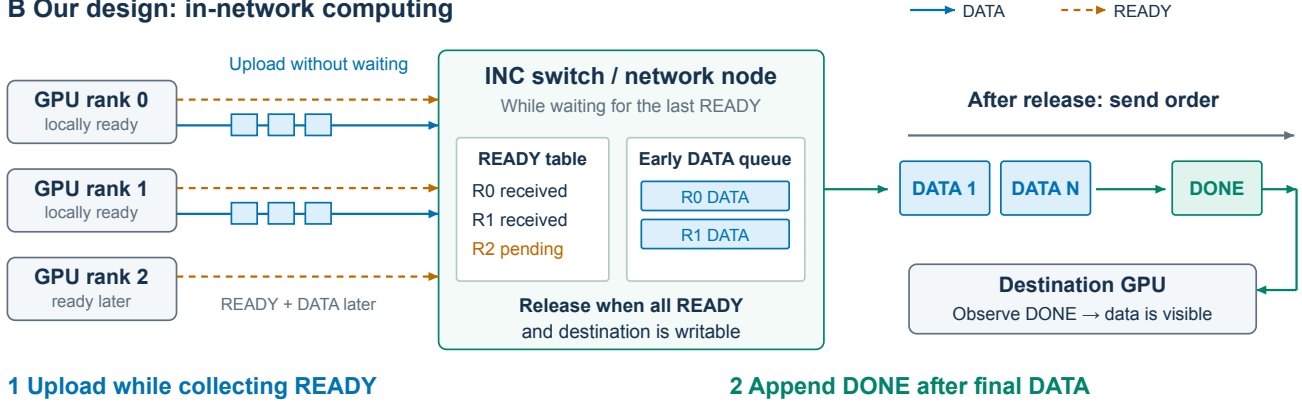


Figure 1. Two changes to Direct coordination: upload data while collecting READY, then append ordered DONE after the destination’s final data. The queue shows the waiting stage; the right-hand sequence shows transmission after forwarding starts. Destination writability, data visibility, and buffer-lifetime requirements remain.

The design applies within or across nodes if all relevant traffic passes through INC and data-before-DONE ordering holds through to the destination GPU. With multiple forwarding stages or network rails, completion tracking must include every stage.

3.2 Readiness Aggregation and Early Upload

After computing its routes and local token counts, rank s sends READY with its buffer epoch and a count vector. Element d specifies how many token copies it will send to destination d . The rank then uploads DATA without waiting for other ranks. INC accepts one complete READY/count-vector message per rank and round, ignores duplicates and old messages, and sums the vectors as they arrive.

Once all required READY messages arrive and destination buffers are writable, INC reads each token’s route and forwards it to the specified destinations. Forwarding can start before the vector sum finishes; INC needs the final count only when deciding to send DONE. Both the announced counts and the forwarding counters count the token copies sent to each destination.

This allows upload to overlap waiting for other ranks. The latency reduction depends on rank readiness times and network queuing (Section 4.2). Global receive counts and final expert-buffer offsets can be computed while tokens enter the preallocated receive buffers. Hardware can pipeline the vector additions with transfer, although any unfinished additions delay DONE.

3.3 Completion Notification

For Dispatch round g , let $n_{g,s,d}$ be the token count sent by source s for destination d . INC sums the count vectors to obtain the expected receive count:

$$N_{g,d} = \sum_s n_{g,s,d}. \quad (1)$$

Every source supplies a vector, including destinations for which it sends no tokens. INC also counts forwarded tokens, incrementing the counter once per token copy after its complete payload has been submitted for delivery.

INC sends $\text{DONE}(g, d)$ when (i) it has summed the count vectors from all required sources, (ii) destination d is ready, and (iii) it has submitted all $N_{g,d}$ token payloads for delivery. DONE must become visible only after those payloads are visible in destination memory:

$$\text{observe}_d(\text{DONE}(g, d)) \Rightarrow \bigwedge_{x \in \mathcal{D}_{g,d}} \text{visible}_d(x), \quad (2)$$

where $\mathcal{D}_{g,d}$ denotes the expected input at d . DONE confirms that this input is available; the destination must still satisfy any other dependencies before computation.

For Combine, INC uses the expert assignments recorded during Dispatch to determine which outputs are expected for each token. Each expert output carries its top- k slot index, distinguishing experts on the same rank. INC sums the weighted expert outputs [16, 18] and sends DONE after all expected outputs have arrived and all reduced results for the

destination have been submitted for delivery, with the same ordering guarantee.

3.4 Buffer Management and Progress

READY confirms that the receive buffer is available for the current round. The buffer remains reserved until its last reader finishes; DONE confirms data arrival, not permission to overwrite. Source buffers are retained until the sender finishes reading them. Generation and buffer-epoch checks reject stale signals. Generation and buffer-epoch checks reject stale signals.

INC uses PFC [8] to pause DATA before token buffers or Combine partial-sum storage fill up, reserving headroom for data already in flight. Transmission resumes when space becomes available. READY and count vectors use a separate traffic class with reserved resources, preventing DATA pauses from blocking the signals needed to start forwarding. Backpressure protects finite storage but can reduce overlap or throughput.

4 Latency Analysis

We compare when the baseline and INC make the required input available for the next computation. The comparison includes rank readiness, data delivery, and completion notification.

4.1 Delay Model

Following Swift's separation of endpoint and network delay [9], we distinguish the time spent issuing a message, transporting it through the network, and observing it at the receiver:

$$T_{\text{msg}} = T_{\text{sender}} + T_{\text{network}} + T_{\text{receiver}}. \quad (3)$$

Here, T_{sender} is the time to issue the send, T_{network} includes link propagation, packet transmission, and network queuing, and T_{receiver} is the time to detect the received message.

For a barrier, each rank waits until all required signals have arrived and local synchronization has finished. Its duration therefore depends on both message delay and when other ranks send their signals; it is not simply T_{network} .

4.2 Overlapping the Pre-Barrier

Let r_i be the time rank i becomes locally ready, and $\mathcal{R}_i$ the peers it waits for. Let $u_{j,i}$ include the time to issue and deliver rank j 's READY to i , and v_i the time for i to observe the signals and finish its local synchronization. The baseline starts sending data at

$$g_i = \max \left\{ r_i, \max_{j \in \mathcal{R}_i} (r_j + u_{j,i}) \right\} + v_i. \quad (4)$$

With INC, rank i can start uploading at $r_i + \delta_i$, where δ_i is its READY and upload preparation time, provided PFC has not paused the link. Let a_i be the time to send its complete

READY/count-vector message to INC, and λ_d the time destination d 's buffer becomes writable. INC can start forwarding to d at

$$o_d = \max \left\{ \max_i (r_i + a_i), \lambda_d \right\}. \quad (5)$$

Here i ranges over the required participants. READY confirms buffer readiness; buffer addresses are fixed during initialization. INC does not wait for the count-vector sum before forwarding.

The difference is that data can already be traveling to INC while READY messages are being collected. Once forwarding starts, buffered tokens compete for the output link with newly arriving tokens. Output queuing, late ranks, and PFC pauses determine how much of the upload overlap reduces final delivery time. Figure 2 illustrates both unequal and simultaneous readiness.

4.3 Overlapping the Post-Barrier

In Direct, ranks finish the required sender-side work before exchanging post-barrier signals [4, 14]. Let c_j be the time rank j can issue its signal, $w_{j,d}$ the time to issue and deliver it to d , and v_d the final observation time at d . The destination observes all required signals at

$$z_d = \max_{j \in \mathcal{R}_d \cup \{d\}} (c_j + w_{j,d}) + v_d. \quad (6)$$

The set includes the destination's own local completion, with no network propagation for that term. Let τ_d be the time all required data is visible in its memory. The baseline communication is complete at $\max(\tau_d, z_d)$.

For INC, let τ'_d be the corresponding data-visibility time and η_d the remaining delay until DONE is observed. This includes any unfinished count aggregation and the cost of delivering an ordered notification. The difference in communication completion times is

$$\Delta T_{\text{tail},d} = \max(\tau_d, z_d) - (\tau'_d + \eta_d). \quad (7)$$

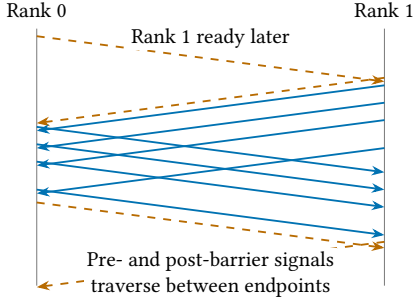
If data arrives at the same time in both designs, INC saves the post-barrier wait remaining after data visibility, less its notification delay. Changes in data delivery, forwarding across multiple paths, and notification processing must also be included. Any additional computation dependencies remain unchanged.

The measured post-barrier window in Section 5.3 runs from signal initiation to signal observation. It does not locate τ_d , so that entire window cannot be substituted for the saving above.

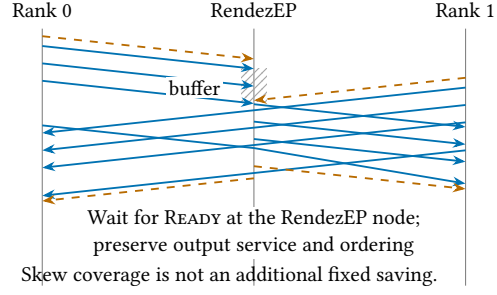
Ideal reference. Consider simultaneous readiness, one-way network delay L , payload transmission duration S , and unchanged local processing time P . Suppose the baseline exposes one separate control traversal before data and another after data becomes visible:

$$T_{\text{base}} = P + L + (S + L) + L = P + S + 3L. \quad (8)$$

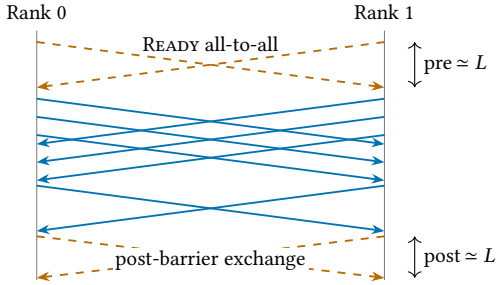
(a) Unequal ranks: endpoint exchanges



(b) Unequal ranks: bounded staging



(c) Aligned ranks: endpoint exchanges



(d) Aligned ranks: RendezEP overlap

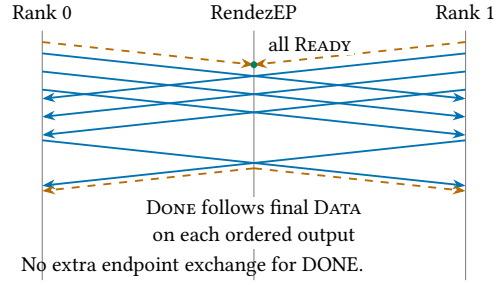


Figure 2. Idealized projection of the two-boundary design, not a measured Direct trace. Solid arrows carry DATA; dashed arrows carry control. The left-hand reference assumes an exposed tail-signal interval of approximately L after data arrival; actual Direct timing follows Equation (7). RendezEP overlaps readiness with upload and appends ordered DONE at each destination’s tail. Early data is buffered under skew. Local transfer completion and safe buffer reuse remain required.

If INC receives READY before the following data, sustains the same forwarding rate, and finishes counting before notification is needed, upload can overlap the first traversal and DONE can accompany the final data delivery. With additional INC overhead ϵ ,

$$T_{\text{INC}} = P + S + L + \epsilon, \quad (9)$$

$$\Delta T = 2L - \epsilon. \quad (10)$$

Thus, the ideal opportunity is one network RTT before additional overhead. Unequal readiness, queuing, and backpressure must instead be evaluated using the actual data and notification times; measured pre-barrier and post-barrier durations are not directly additive savings.

5 Evaluation

We measure synchronization in the baseline Direct path and its contribution to MoE inference. The experiments establish the size and location of the coordination windows used to motivate the design. They do not measure RendezEP performance.

5.1 Experimental Setup and Methodology

Each measurement node has eight H200 GPUs connected by NVSwitch and eight 400 GbE RoCE interfaces. Cross-node experiments use GIN’s GDAKI backend and traffic class 162.

The software stack uses PyTorch 2.13, CUDA 13, NCCL 2.31.2, and DeepEP V2 based on commit 01dc3aaa [4]. Hybrid is disabled. Local communication available within Direct is retained.

Microbenchmarks use $H = 7168$, $\text{topk} = 8$, 256 experts, BF16, and balanced uniform routing. T denotes source tokens per rank. Each point has three independent process runs with 200 active iterations per run, except the post-barrier measurements at $T = 32$, which pool two three-run batches. Error bars show standard deviation across process-run means.

Communication paths retain their baseline algorithms with measurement instrumentation. Total operator latency uses trace-off runs. Phase durations and the denominators for phase shares come from the same trace-on runs. For each iteration and phase, we select the rank with the shortest observed phase and divide by that rank’s operator span. We then average the iteration shares and the run means. This is a rank-minimum observation, not a measurement of an ideal barrier with simultaneous arrivals; it can still contain peer waiting. Independently selected phases are not additive.

The pre-barrier share sweep and the post-barrier signal-window sweep were acquired in separate campaigns on the post-stage ABI image. The pre-barrier constants across EP sizes use the original campaign, supplemented by the EP8

EP	Fitted T	Dispatch	Combine	Max. dev.
4	8/32/96/128	11.65	11.54	0.06
8	8/32/64/96/128	14.96	14.94	0.16
16	8/32/96/128	18.36	18.28	0.09

Table 1. Pre-barrier constants and maximum deviation of per- T run means, in μ s. Each T has three process runs. EP8 additionally uses the later, differently instrumented $T = 64$ batch.

$T = 64$ measurements. Instrumentation and run-order effects are not assumed to vanish. We preserve per-run values and configuration provenance and do not infer speedups by subtracting trace-on phases from trace-off totals.

5.2 Pre-Barrier

Figure 3 reports pre-barrier duration and share at EP8. Both operators remain near 15μ s at all four measured token counts. Their shares fall from 20.57% and 19.84% at $T = 8$ to 4.88% and 5.05% at $T = 128$. The fixed pre-barrier cost consequently matters most when payload work is short.

We also summarize the pre-barrier durations at EP4, EP8, and EP16 with a separate constant for each configuration (Table 1). All available small-batch points listed in the table participate in the estimate, including $T = 96$ and the later EP8 $T = 64$ point. The largest departure of a token-count mean from its configuration’s constant is 0.16μ s. This is an in-sample description, not a held-out prediction.

The fitted constants increase across the measured configurations. However, EP4/8/16 also change the number of GPUs per node, traffic destinations, and network concurrency. These three estimates do not establish a general dependence on rank count.

5.3 Post-Barrier

The original enclosing timer, named post-barrier in the instrumentation, can start when an early warp finishes its work, while other warps still transfer data. It therefore includes payload work and completion waits. We report the post-barrier signal window: from signal initiation, after the local flush and join, to observation of all expected signals. This window includes GPU initiation and polling as well as possible waiting for peers.

Figure 4 uses the repeated four-point campaign and subsequent $T = 16, 32, 96$ measurements. At $T = 8$ and 16, Dispatch is approximately 14μ s and Combine is 13.6 – 13.8μ s. The measured means at $T \geq 32$ lie around 11 – 12μ s. Each of these points has three runs, with six retained runs at $T = 32$.

The $T = 32$ sample has substantial variation. One batch includes a run with Dispatch and Combine means of 13.51 and 14.44μ s; its timestamps passed validation, so we retain it. Pooling both batches gives $11.67 \pm 0.94 \mu$ s and $11.70 \pm 1.43 \mu$ s, respectively. The repeated measurements do not support a

smooth token-count formula, and the unmeasured interval between $T = 16$ and 32 prevents identification of a transition point.

For fixed EP8, the inspected tail-signal loop sends to the same rank set regardless of T . Payload work can change when ranks enter that loop, so peer readiness, endpoint scheduling, and network queuing are plausible sources of variation. The measurements do not separate these effects. We report the observed durations without interpreting either a protocol switch or a removable INC delay.

5.4 Synchronization in MoE Inference

We measure Qwen3-30B-A3B in BF16 with vLLM 0.27 and DeepEP V2 Direct. The model has $H = 2048$, 128 experts, and $topk = 8$. The serving setup uses actual model routing, TP=1, and EP8; requests have 128 input and 128 output tokens. We compare one node with eight ranks to two nodes with four ranks each at concurrency 64 and 128.

The serving build includes measurement instrumentation and an expert-padding correctness fix. The sampled decode step is checked for 48 Dispatch and 48 Combine trace entries per active rank. Pre-barrier intervals are paired with each rank’s own GPU step, then averaged over ranks and three client windows. The windows share a server instance; they are not independent server restarts.

At concurrency 64, the measured pre-barrier share is 9.33% on one node and 16.72% across two nodes. At concurrency 128, it is 7.94% and 15.19%, respectively (Figure 5). Pre-barrier remains a substantial part of the observed cross-node decode step, even with real routing and the other work of the serving stack.

These shares use GPU step time, not request latency. Their aggregation also differs from the microbenchmark rank-minimum statistic, and the model’s hidden size and expert count differ from the microbenchmark. We do not combine their absolute times or convert the shares to a speedup. The post-barrier signal window was not instrumented per layer in the serving experiment, so its end-to-end share remains unmeasured.

The current evidence establishes baseline coordination costs. Measuring the design’s benefit requires an implementation and observations of destination data visibility and the start of dependent computation, as distinguished in Equation (7).

6 Related Work

6.1 EP Communication

DeepEP [3], NCCL EP [5], and GPU-initiated networking [6] provide the communication paths and transport primitives considered here. SwiftEP [10] optimizes buffer movement and GPU work, while COMET [19] overlaps communication and computation. These optimizations affect how payload

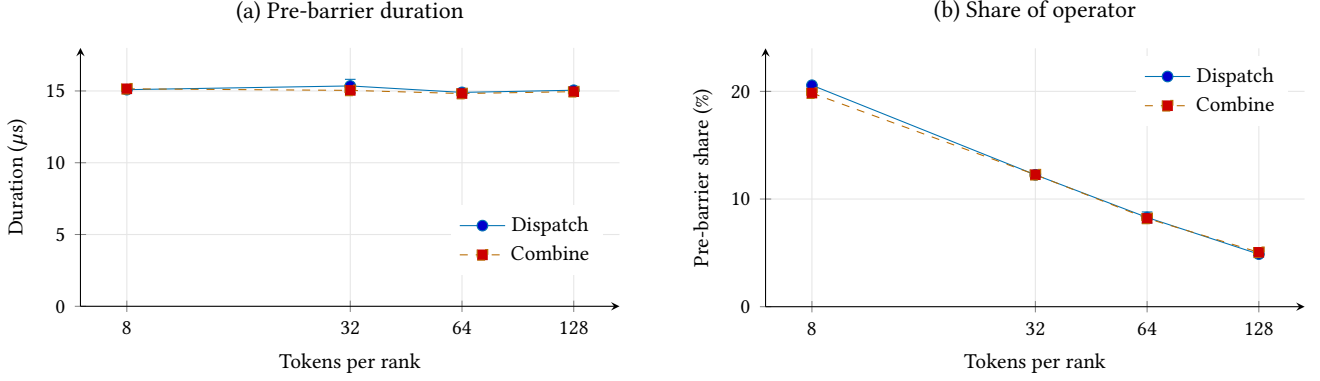


Figure 3. H200 cross-node EP8 Direct. Pre-barrier duration stays close to 15 μs while its operator share decreases. Both use the selected rank’s trace-on measurements; error bars are standard deviations across three run means. Lines connect measured points.

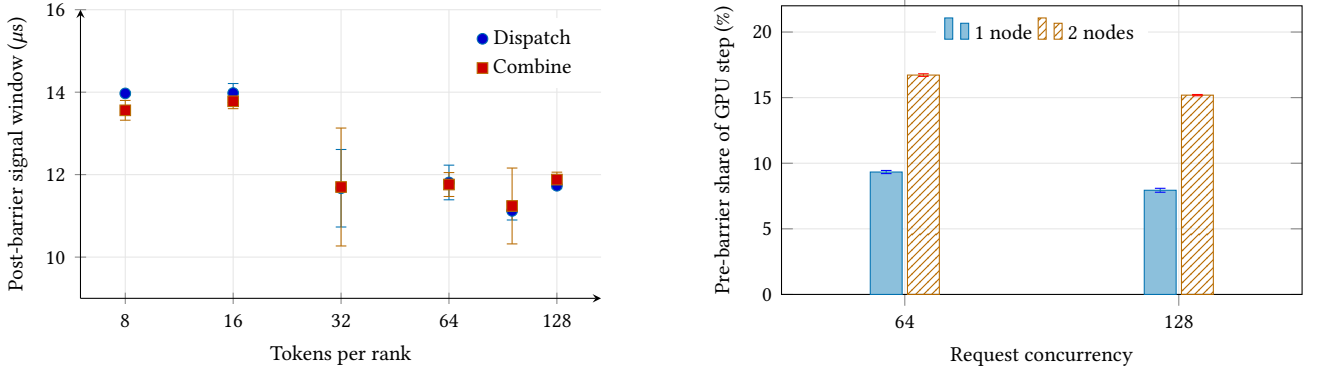


Figure 4. EP8 post-barrier signal-window observations, without a fitted curve. Each point has three runs except $T = 32$, which pools six runs from two batches. Error bars show between-run standard deviation.

Figure 5. Qwen3-30B-A3B decode at fixed EP8. Pre-barrier intervals for Dispatch and Combine are paired with the same rank’s GPU step. Error bars describe three client windows, not server restarts.

moves and when computation can proceed. Our design targets readiness aggregation before delivery and completion notification after managed output.

Perseus [15] reorganizes PUT, fence, and signal processing within the transfer pipeline. UBEP [11] uses point-to-point signals and Data-as-Flag to reduce stage synchronization in a scale-up EP implementation. Both motivate examining the dependencies enforced by existing synchronization. RendezEP instead considers an explicit network rendezvous with bounded early-data staging and destination notification. These systems differ in how they order writes and manage buffers.

6.2 In-Network Computation

DySHARP [18] combines dynamic addressing, multicast, and reduction in the network for MoE. MultiWrite [2] reduces duplicate upstream traffic through multicast writes. Their

data-plane techniques can complement the proposed coordination. We do not claim multicast arithmetic or bandwidth reduction as a new contribution.

SHARP [17] and SwitchML [16] demonstrate network-assisted collectives and reduction state management. They provide useful precedents for the Combine data plane. The question here is how readiness aggregation and output completion change the timing of an EP operation with routing-dependent traffic.

6.3 Synchronization Offload

EPIC [1] provides protocol and resource abstractions for Ethernet in-network computing. Such interfaces are relevant to a possible implementation of RendezEP, which requires programmable progress tracking, bounded storage, and data-before-DONE ordering. GPU-initiated networking [6] exposes the endpoint PUT, signal, and fence operations used

by our baseline; the proposed node must coordinate with these transport semantics.

Swift’s endpoint/fabric decomposition [9] informs the latency model, while tools such as FabricPerf [7] motivate explicit observation boundaries. Our measured signal window characterizes endpoint coordination. It does not by itself identify the fabric propagation or the delay exposed after destination visibility.

7 Conclusion

Cross-node DeepEP V2 Direct uses a pre-barrier before payload injection and a post-barrier after sender-side completion work. H200 measurements show that pre-barrier remains near 15 μ s at EP8 across the measured small-batch points and occupies 15.2–16.7% of the sampled cross-node MoE decode step. RendezEP proposes to overlap readiness aggregation with upload and to generate ordered completion notification at the network output. Its benefit depends on data visibility, safe buffer reuse, finite buffer capacity, and the delay exposed on the consumer’s critical path. The protocol and model describe this opportunity; evaluating its net performance requires an implementation.

References

- [1] 2026. EPIC: Programmable Protocols and Resource Abstractions for Ethernet In-Network Computing. arXiv:2605.18683. <https://arxiv.org/abs/2605.18683>.
- [2] 2026. MultiWrite: Eliminating Redundant Upstream Traffic in Many-to-Many Communication. arXiv:2605.22428. <https://arxiv.org/abs/2605.22428>.
- [3] DeepSeek-AI. 2026. DeepEP: An Efficient Expert-Parallel Communication Library. <https://github.com/deepseek-ai/DeepEP>.
- [4] DeepSeek-AI. 2026. DeepEP V2 synchronization and kernel sources. Commit 01dc3aaac820. Files: common/comm.cuh, impls/hybrid_dispatch.cuh, impls/hybrid_combine.cuh. https://github.com/deepseek-ai/DeepEP/tree/01dc3aaac82068020353dce2c302e38153c0bfaa/deep_ep/include/deep_ep.
- [5] Amos Goldman, Nimrod Boker, Maayan Sheraizin, et al. 2026. NCCL EP: Towards a Unified Expert Parallel Communication API for NCCL. arXiv:2603.13606. <https://arxiv.org/abs/2603.13606>.
- [6] Khaled Hamidouche, John Bachan, Pak Markthub, Peter-Jan Gootzen, Elena Agostini, Sylvain Jeaugey, Aamir Shafi, Georgios Theodorakis, and Manjunath Gorentla Venkata. 2025. GPU-Initiated Networking for NCCL. arXiv:2511.15076. <https://arxiv.org/abs/2511.15076>.
- [7] Songlin Huang and Chenshu Wu. 2026. FabricPerf: Measuring NIC-less Scale-Up Network through GPU Communication Kernel Profiling. Project repository. <https://github.com/open-neutrino/fabricperf>.
- [8] IEEE 802.1 Working Group. 2011. IEEE 802.1Qbb: Priority-based Flow Control. <https://1.ieee802.org/dcb/802-1qbb/>.
- [9] Gautam Kumar, Nandita Dukkupati, Keon Jang, Hassan M. G. Wasel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *ACM SIGCOMM*. 514–528. <https://doi.org/10.1145/3387514.3406591>.
- [10] Xingyi Li, Yadong Liu, Xiaojie Huang, et al. 2026. SwiftEP: Accelerating MoE Inference with Buffer Fusion and TMA Offloading. In *USENIX NSDI*. <https://www.usenix.org/conference/nsdi26/presentation/li-xingyi>.
- [11] Yipeng Liu, Chang Liu, Si Shen, et al. 2026. UBEP: Re-architecting Expert Parallelism Communication Library for Production Superpods. arXiv:2607.06202. <https://arxiv.org/abs/2607.06202>.
- [12] NVIDIA. 2026. NCCL EP HT device implementation. Commit 7b83616df3ae. https://github.com/NVIDIA/ncccl/blob/7b83616df3ae082a1f32bb74c27458bfe8153a13/contrib/ncccl_ep/device/hybrid_ep.cuh.
- [13] NVIDIA. 2026. NCCL EP LL device implementation. Commit 7b83616df3ae. https://github.com/NVIDIA/ncccl/blob/7b83616df3ae082a1f32bb74c27458bfe8153a13/contrib/ncccl_ep/device/ll_ep.cuh.
- [14] NVIDIA. 2026. NCCL GDAKI flush and completion-queue waiting. Commit 7b83616df3ae, gin_gdaki.h. https://github.com/NVIDIA/ncccl/blob/7b83616df3ae082a1f32bb74c27458bfe8153a13/src/include/ncccl_device/gin/gdaki/gin_gdaki.h.
- [15] Byungsoo Oh and Rachee Singh. 2026. Eliminating Hidden Serialization in Multi-Node Megakernel Communication. arXiv:2605.00686. <https://arxiv.org/abs/2605.00686>.
- [16] Amedeo Sapia, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtarik. 2021. Scaling Distributed Machine Learning with In-Network Aggregation. In *USENIX NSDI*. 785–808. <https://www.usenix.org/conference/nsdi21/presentation/sapia>.
- [17] Scot Schultz. 2024. Advancing Performance with NVIDIA SHARP In-Network Computing. <https://developer.nvidia.com/blog/advancing-performance-with-nvidia-sharp-in-network-computing>.
- [18] Qijun Zhang, Chen Zhang, Zhuoshan Zhou, et al. 2026. Accelerating MoE with Dynamic In-Switch Computing on Multi-GPUs. arXiv:2605.05607. <https://arxiv.org/abs/2605.05607>.
- [19] Shulai Zhang, Ningxin Zheng, Haibin Lin, et al. 2025. Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts. arXiv:2502.19811. <https://arxiv.org/abs/2502.19811>.